

Ingeniería del Software 3

Clase 7 · La imagen que se ejecuta, y las pruebas que deciden si se promueve

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

De dónde partimos: la cadena del TP6



Hoy la cadena no cambia: cambia LO QUE VIAJA por ella. Pasa de ser el commit a ser la imagen.

El recorrido de este video

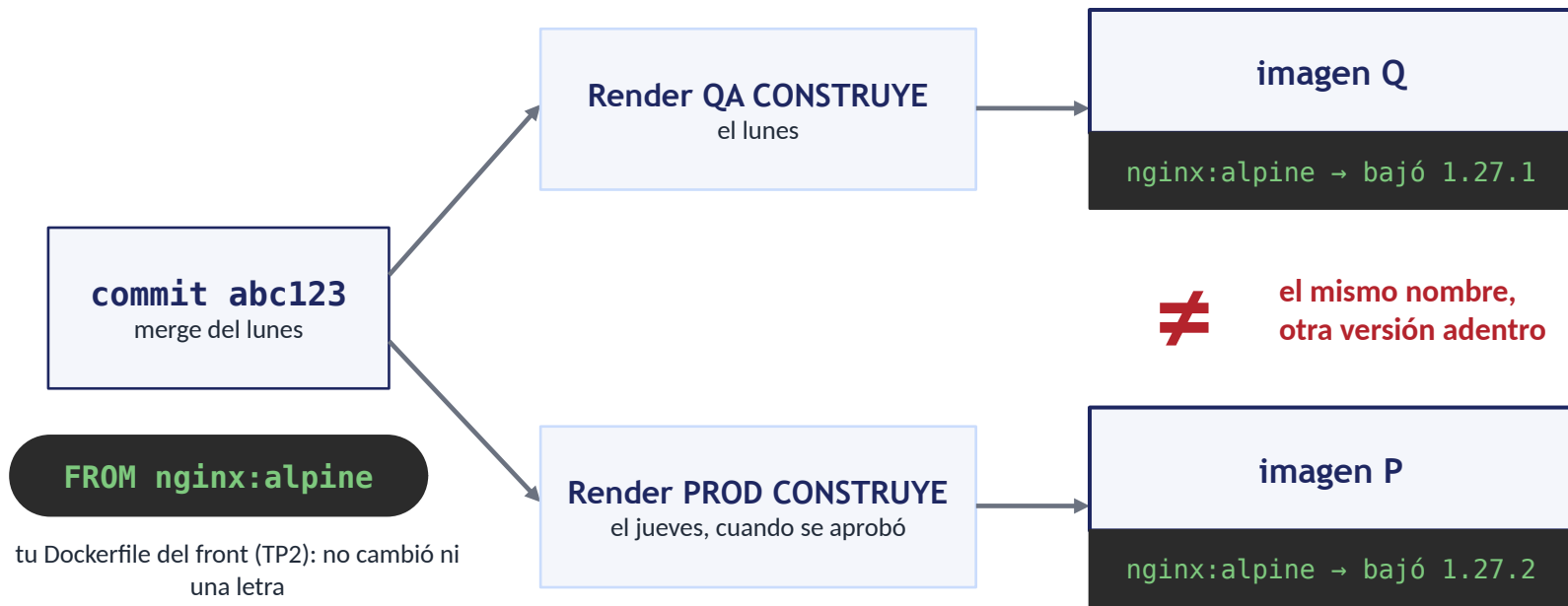
1 · qué imagen corre en cada entorno

2 · cómo lo comprobás desde afuera

3 · las dos pruebas que faltan

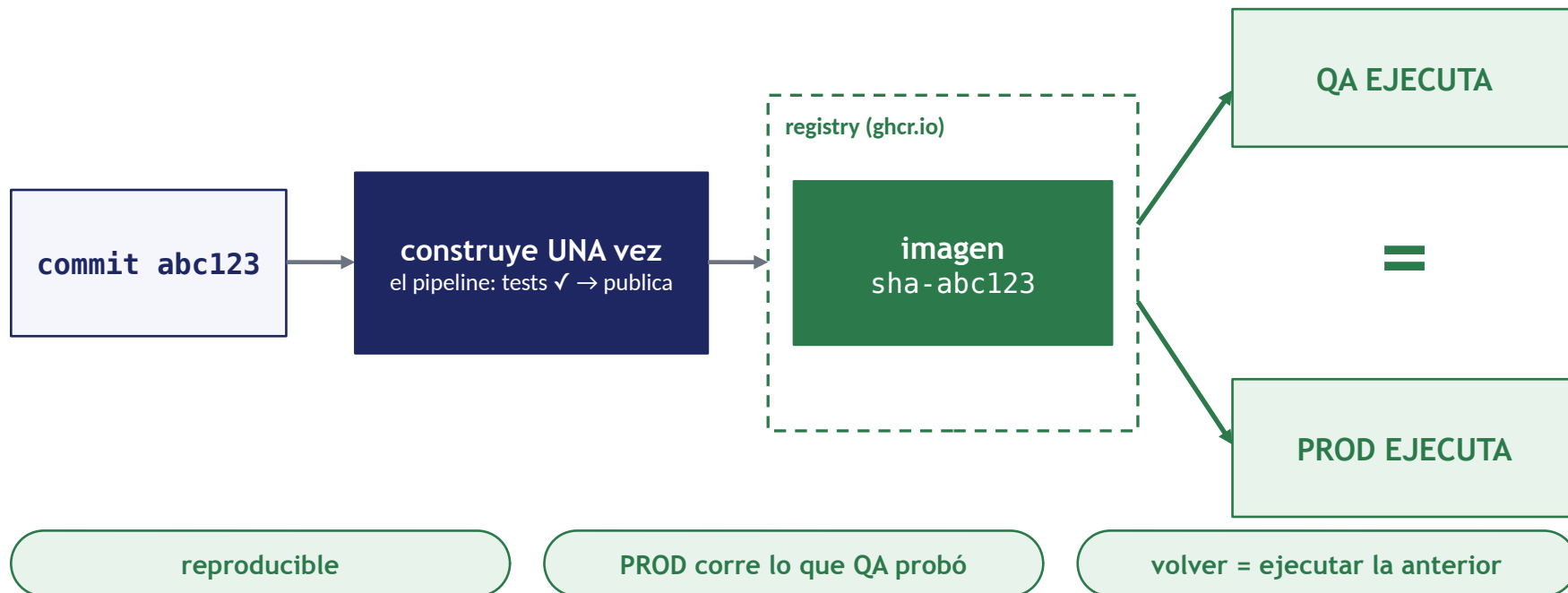
Requisito: esta cadena andando, de punta a punta. No hay cuentas nuevas: el registry, Render y Neon ya los tenés.

El problema: cada entorno vuelve a construir



Lo que QA probó NO es lo que corre en PROD · el mismo código, dos construcciones

Construir una vez, desplegar muchas



La imagen ya la publicás desde el TP6. Lo que cambia es QUIÉN la ejecuta.

La dirección de una imagen

ghcr.io	/<owner>	/<tu-repo>-backend	:sha-abc123...
----------------	-----------------------	---------------------------------	-----------------------

dónde vive: el registry

de quién es: tu usuario

qué imagen

CUÁL VERSIÓN = la etiqueta

La etiqueta, entera:

sha-

abc1234f9c2ab07d5e8a1b3c6d9e0f2a4b6c8d0e

el prefijo: lo pone el pipeline

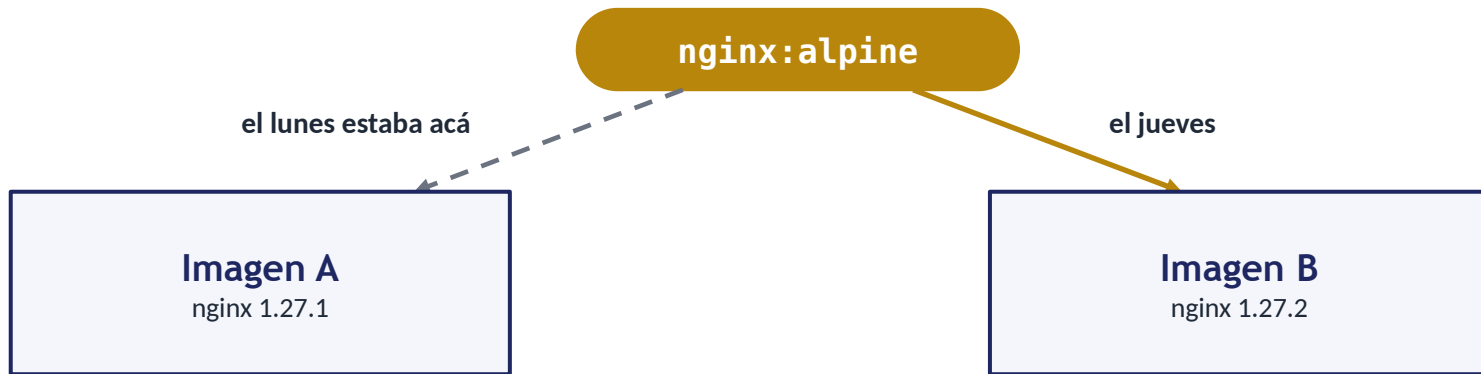
el commit COMPLETO: los 40 caracteres que te da git

git completa el resto
git show abc123 ✓

el registry compara el texto ENTERO
docker pull ...-backend:sha-abc123 x manifest unknown
docker pull ...-backend:sha-<los 40> ✓

Una etiqueta es un TEXTO: o coincide letra por letra, o para el registry no existe. Los 40 salen de github.sha en el pipeline, o de git rev-parse HEAD a mano.

Una etiqueta es un rótulo: se puede mover



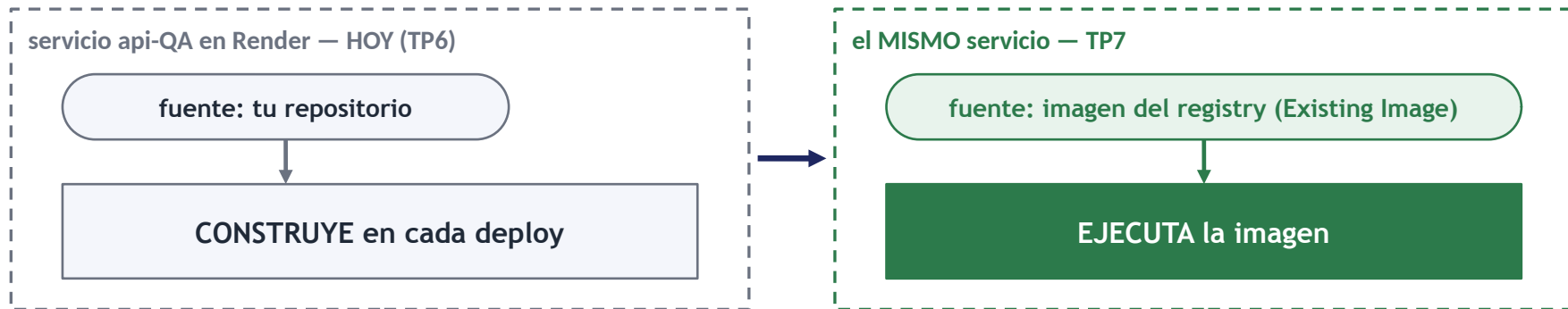
Una ETIQUETA es un rótulo: se despegas y se pegas en otra imagen. Es lo que pasó entre el lunes y el jueves

La tuya la pega tu pipeline, y nadie más la mueve: por eso sirve para promover — nombra siempre la misma imagen

Nunca se promueve por latest, un rótulo que se mueve en cada merge: por eso tu pipeline ni lo publica.

¿Hay algo que no se pueda mover? Sí: el digest (sha256:...), la huella del contenido. En este TP no hace falta que lo uses.

El servicio cambia de fuente, no se recrea



Al cambiar la fuente, el servicio conserva:

la URL

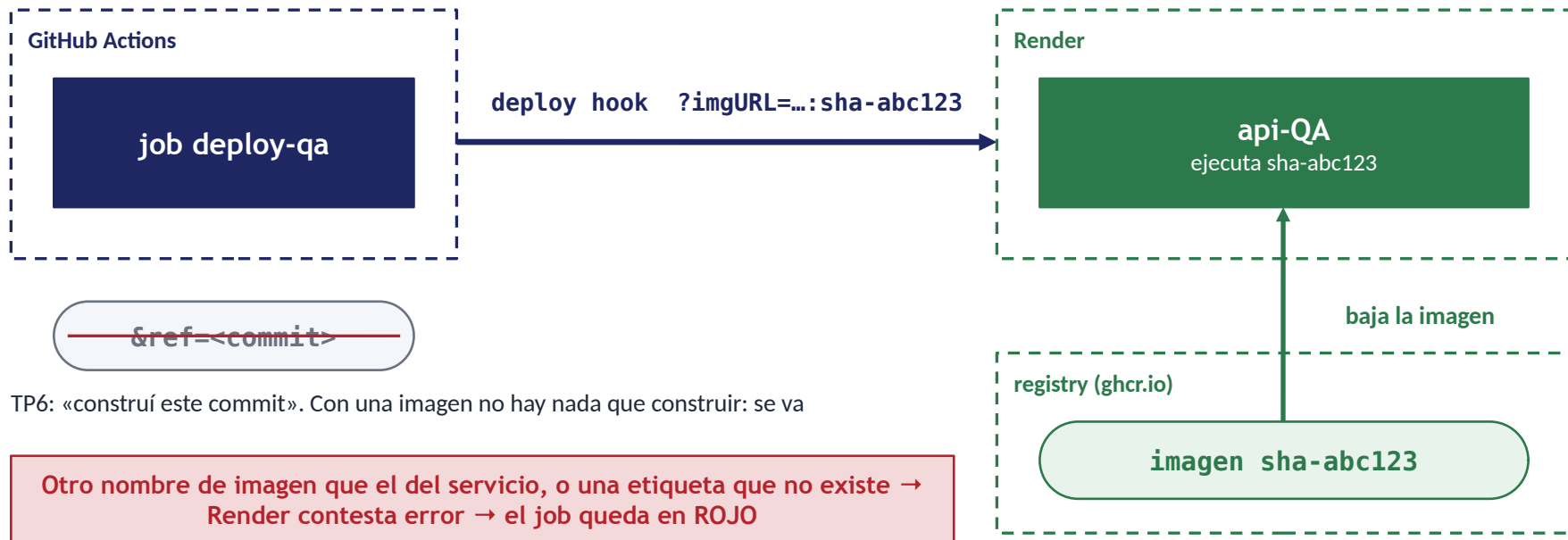
las variables

el deploy hook

el historial

¿Y por qué no crear servicios nuevos? Servicio nuevo = URL nueva y deploy hook nuevo: hay que recargar los secrets de los hooks, BACKEND_URL, el smoke y las pruebas. Si se te escapa una, todo puede seguir dando verde, pero contra los servicios VIEJOS.

Quién le dice qué imagen: cada deploy, con imgURL



```
curl -fsS --get --data-urlencode "imgURL=$IMG_API" "$HOOK_API" # antes: ...&ref=$GITHUB_SHA
```

QA y PROD reciben el MISMO nombre de imagen: «se promueve lo que se verificó» queda escrito en la llamada

Cómo se cambia la fuente: dos momentos

① UNA VEZ por servicio • a mano, en el panel de Render

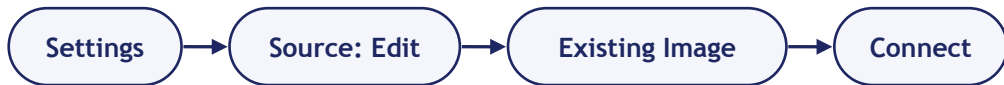


Image URL = la imagen de tu ÚLTIMO merge, con los 40:

```
ghcr.io/<owner>/<tu-repo>-backend:sha-<los 40>
```

en los dos fronts: ...-frontend, con la misma etiqueta

¿De dónde salen los 40? En main, después del merge:

```
git pull && git rev-parse HEAD
```

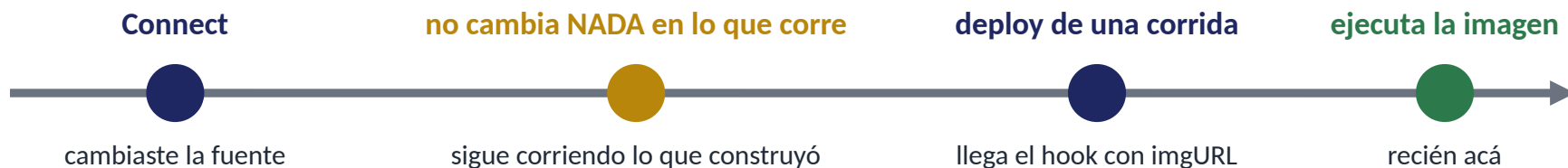
② CADA deploy • tu pipeline

deploy hook + imgURL
la imagen de ESA corrida

de acá en más, el deploy lo dispara siempre
tu pipeline

Lo de ① es sólo el punto de partida: lo que corre lo decide ②, en cada corrida.

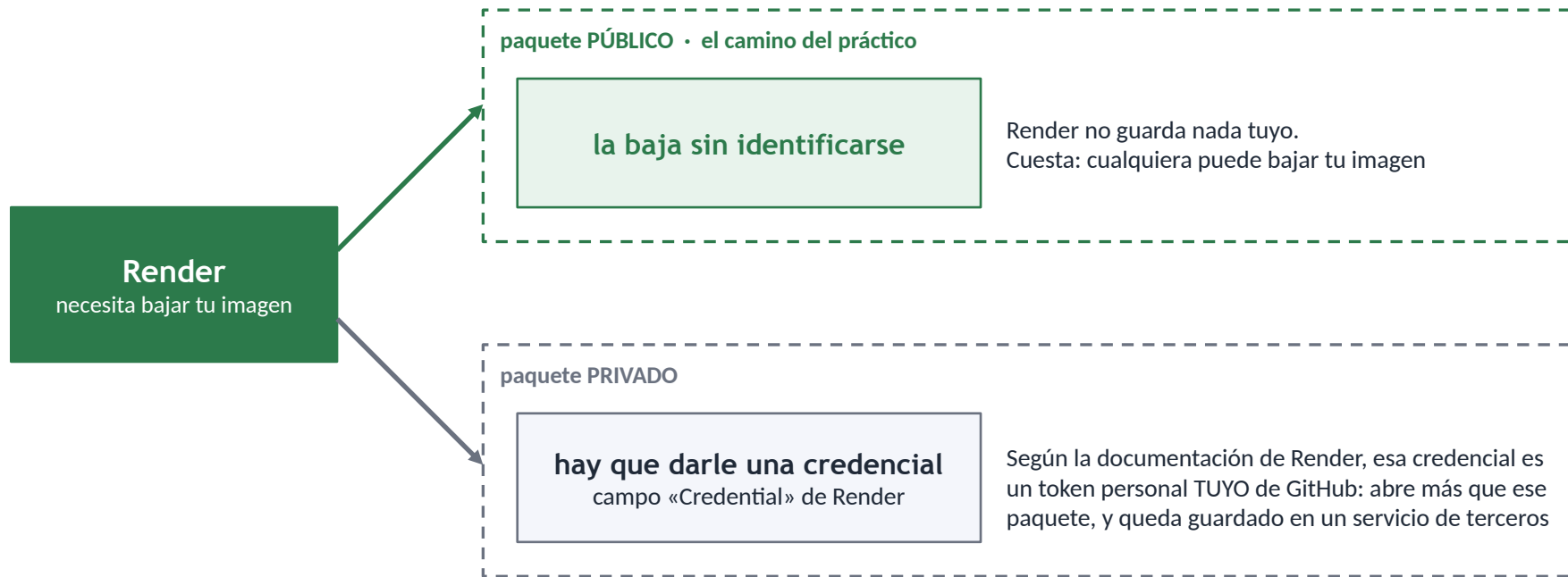
Tocaste Connect: qué pasa después



Cambiar la fuente NO despliega. No falló, y tampoco está listo: el servicio sigue con lo último que construyó desde el repositorio, hasta que le llega el hook con imgURL.

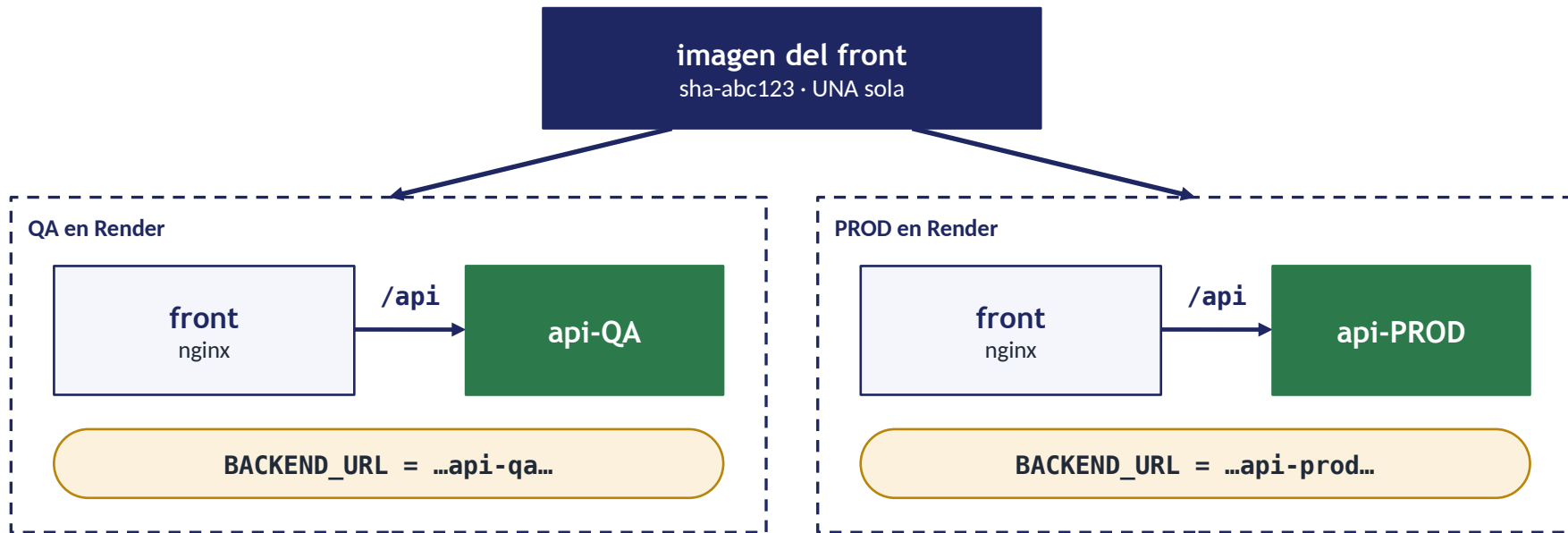
Primero UN servicio, de QA: si algo falla, falla en uno, y no en producción. Cuando anda, los otros tres.

¿Cómo baja Render tu imagen? Paquetes públicos



Como cualquiera la puede bajar, la imagen NO lleva secretos: contraseñas y direcciones van en las variables del servicio

La misma imagen del front, en QA y en PROD

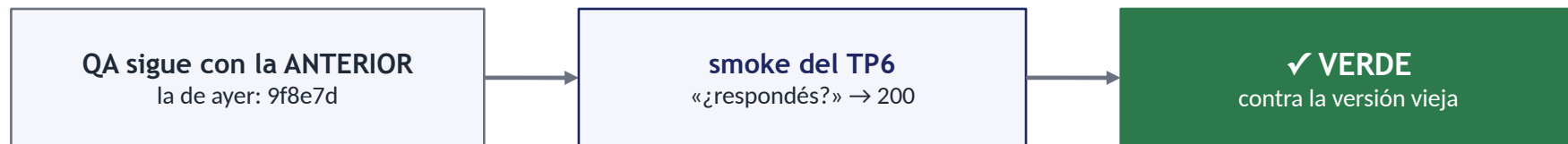


Lo que distingue a un entorno va en VARIABLES del servicio, no adentro de la imagen. Por eso, al cambiar la fuente, las variables no se tocan.

Pedí abc123. ¿Está corriendo abc123?



El caso: el deploy tarda, o falla



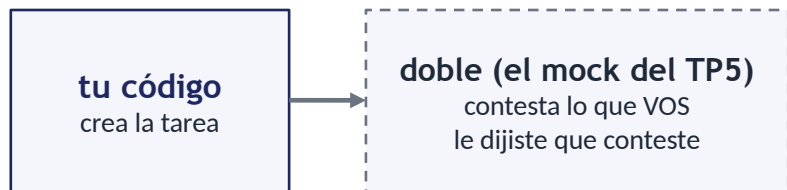
Responde bien, porque la versión de ayer también andaba. Verde no dice QUÉ respondió — y lo que sigue (las pruebas, la aprobación) prueba y promueve otra cosa.

En este TP lo comprobás VOS, en Events —la pestaña de Render que lista cada deploy del servicio—: el deploy de cada servicio nombra la etiqueta sha- de esa corrida. Cerrarlo automáticamente —que la api diga qué commit corre y el smoke COMPARE— es el paso siguiente, y no entra acá.

Integración: lo que el doble no ve

La app de ejemplo: una lista de tareas. Agregar una = POST /api/tareas {"titulo": "comprar pan"} y la api la guarda en la base.

TP5 • unitario: SACASTE una dependencia y pusiste un doble



TP7 • integración: la VOLVÉS A PONER



El doble sirve para probar tu lógica sin depender de nada. Lo que no puede contestar: ¿qué dice la dependencia de verdad? El caso típico: una fecha que tu código arma sin zona horaria, que el doble acepta y Postgres rechaza. La guía lo desarrolla en su §2.5.

Dos formas de volver a ponerla. ESTRECHA: la api armada dentro del pipeline, con una base descartable. • AMPLIA: la api YA DESPLEGADA en QA, con su base. **En este TP, la amplia: mismo concepto, un décimo de la configuración.**

La pirámide entera: qué ve cada escalón

	VE	SE LE ESCAPA	TARDA
E2E · TP7	lo que vive la persona: pantalla, api, base y red, juntas	lo que no recorren tus pocos flujos, y lo que sólo pasa en producción	minutos
Integración de BACK · TP7	tu api DESPLEGADA con su base de verdad	si la pantalla usa bien esa api	segundos
Unitarios · TP5	una regla de tu código, aislada	lo que contesta la dependencia de verdad (pusiste un doble)	milisegundos

Más arriba: ve más, y cuesta más. Por eso arriba van pocas.

¿Por qué recién ahora estas dos? Las dos necesitan un entorno desplegado que corra TU imagen: la integración le habla a su api, la e2e la usa por la pantalla. Desde la primera parte de este video, QA lo es: por eso la cadena va primero.

¿Y la integración de FRONT? Existe: varios componentes TUYOS juntos —el formulario y la lista— con vitest y Testing Library, en el runner, sin navegador ni entorno desplegado. No entra en este TP.

Responde $\neq$ funciona: smoke y e2e

Caso: BACKEND_URL mal cargada en el front de QA: nginx manda /api a una dirección que no es tu api. La api anda; la página no llega a ella.

smoke

GET /health

200 {"status": "ok"}

✓ VERDE

e2e

abrir la página

escribir una tarea

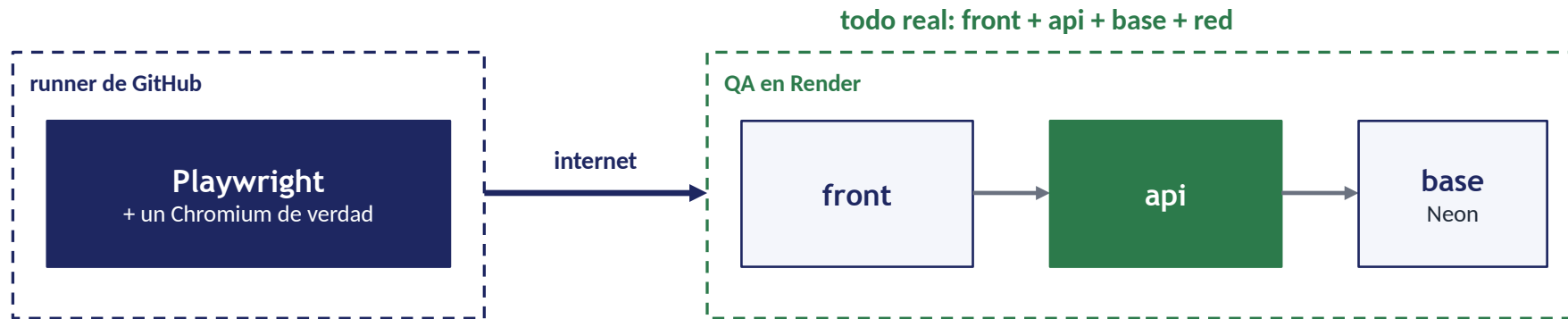
clic en Agregar

¿aparece en la lista?

✗ ROJO: nunca aparece

el smoke pregunta ¿RESPONDÉS? · la e2e, ¿FUNCIONÁS?

Dónde corre una e2e

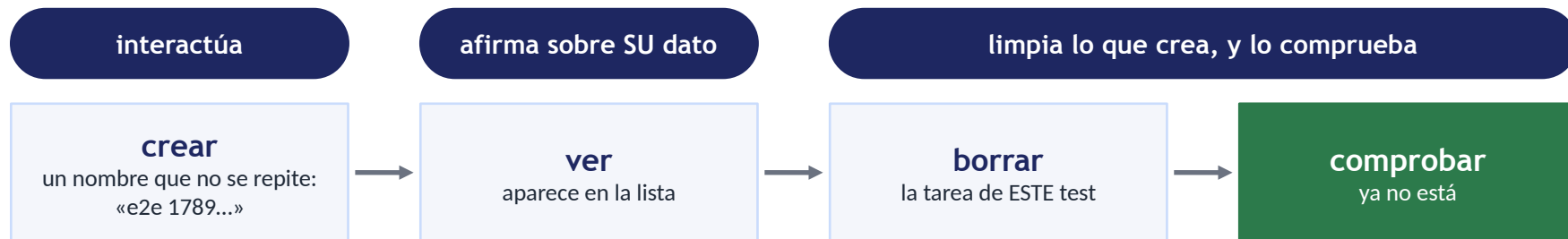


La dirección no está escrita en el test: E2E_BASE_URL le dice a Playwright a cuál apuntar



La lee Playwright, en su playwright.config.js. Ni Render ni el pipeline saben que existe.

Anatomía de una e2e



```
// frontend/e2e/tareas.spec.js
await page.getByLabel('Título de la nueva tarea').fill(titulo)
await page.getByRole('button', { name: 'Agregar' }).click()
await expect(page.getByText(titulo)).toBeVisible() // espera sola: sin sleep
await expect(page.getByText(titulo)).toHaveCount(0) // después de borrar
```

getByLabel · getByRole

buscan como una PERSONA: por el texto que nombra el campo, por el nombre del botón. Cambia el diseño y no se rompen

Mínimo tres flujos

uno de creación (éste)
uno de error: dato inválido → mensaje → no se creó nada
el que tu usuario hace TODOS LOS DÍAS

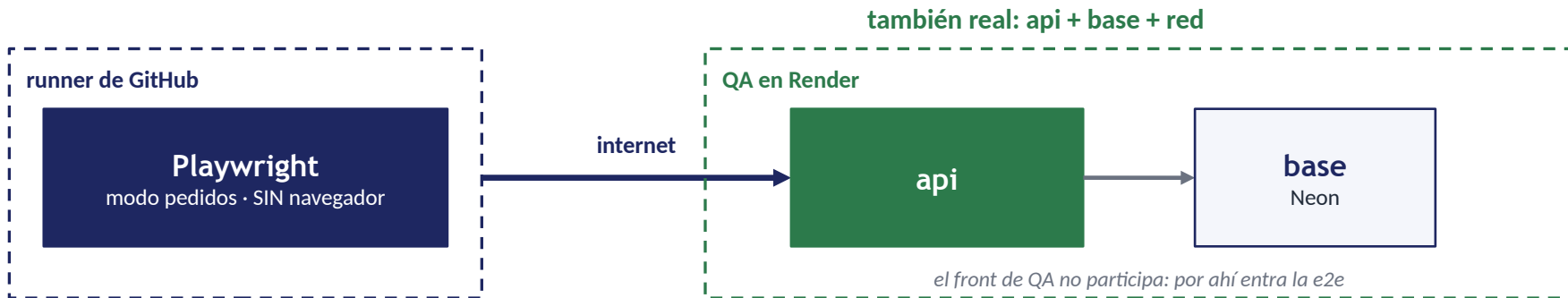
e2e confiables: cuatro reglas

Regla	Por qué
Pocas y críticas	3-5 flujos: si se rompe ESO, la app no sirve. Lo que ya cubren los unitarios no se repite
Contra lo real	Sin respuestas inventadas ni tests que se saltean solos: con un doble de tu api probás tu front contra un backend que no es el tuyo. Es condición de aprobación
Esperan, no duermen	Un sleep a veces alcanza y a veces no: fabrica tests flaky (pasan o fallan sin que cambie nada), y el equipo aprende a ignorar el rojo
Limpian	QA es compartido: si cada corrida deja basura, la décima falla por culpa de la primera

QA gratis DUERME, y por internet un pedido puede demorarse: por eso la config lleva esperas largas y UN reintento. Ojo — ese reintento deja la corrida en VERDE con el test marcado «flaky»: abrí el reporte igual.

La integración, en el pipeline: pedidos sin navegador

¿Y cómo le hablás a la api sin navegador? Con la MISMA herramienta de la e2e, en su otro modo.



```
// e2e/api.spec.js – sin navegador: el fixture `request`
const API = process.env.API_BASE_URL // la API, NO el front
const T = `${API}/api/tareas`
const titulo = `api ${Date.now()}` // único por corrida
const alta = await request.post(T, { data: { titulo } })
expect(alta.status()).toBe(201) // si la base dice no: 500
const todas = await (await request.get(T)).json()
expect(todas.some(t => t.titulo === titulo)).toBe(true)
await request.delete(`${T}/${(await alta.json()).id}`)
```

```
# ci.yml → jobs:
integracion:
  needs: deploy-qa
e2e:
  needs: integracion
```

Si la api está rota,
la e2e ni corre.

● integración + ● e2e = el FRONT · ● integración + ■ e2e saltada = la API o la base. La e2e sola es una alarma. Con la integración al lado, es un diagnóstico.

El gate: las pruebas deciden la promoción

todo en UNA corrida: el needs vuelve obligatorias a las dos pruebas



rojo

```
# ci.yml → jobs:
integracion:
  needs: deploy-qa
e2e:
  needs: integracion
deploy-prod:
  needs: e2e      # ← el freno
```

deploy PROD queda salteado, en gris:
no pide aprobación

El gate no mira sólo tu código: mira que ESTE despliegue quedó bien. Quien aprueba ya no ve «QA responde»: ve «la api contestó bien con su base, y un navegador usó la app entera».

En la MISMA corrida, no en un workflow aparte:

así no depende de que alguien se acuerde de correrla, y de mirarla, antes de aprobar.

Probar el freno: rompé la app, no el test

La rotura: el front y la api dejan de llamar igual a un dato. Las pruebas, sin tocar.

```
antes  POST /api/tareas {"titulo": "comprar pan"} → 201: la tarea aparece
roto   POST /api/tareas {"title": "comprar pan"} → 400, rechazado: la tarea nunca aparece
```

smoke ✓

le pregunta a la api si responde:
contesta 200

integración ✓

le habla a la API con el nombre
bien escrito: guarda y lee

e2e ✗

usa la PANTALLA como una
persona: la tarea nunca aparece



deploy PROD

salteado: no pide aprobación

¿Por qué la app, y no el test?

Un test editado para que falle no dice nada. Lo que querés saber es si
tu test VE una app rota

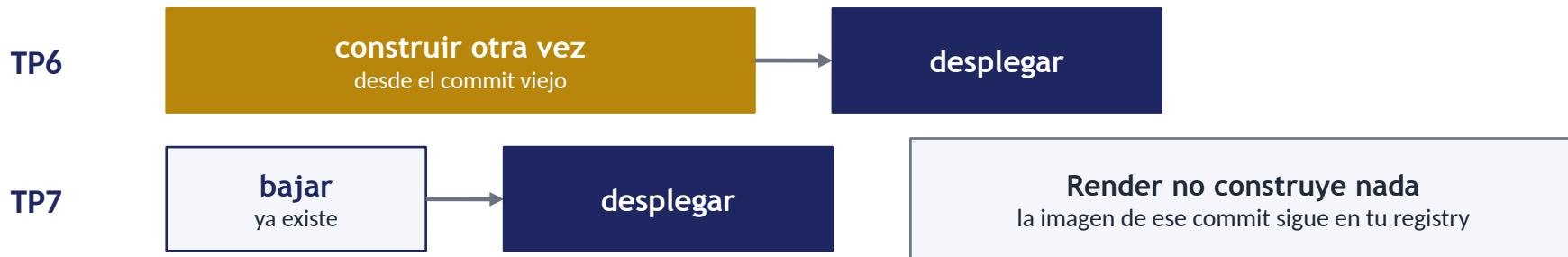
¿Por qué el front, y no la api?

Lo que sólo la e2e ve es cómo el front USA la api. Si rompés la api al
crear una tarea, lo ataja antes la integración —y por el needs, la e2e ni
corre—. Otra que sirve: el texto del botón

La release: el nombre de lo que está en producción

El práctico cierra con su versión, **v7.0.0**, como el **TP6**: un tag de git sobre el commit que está en producción —no sobre el último de main—. Es el NOMBRE al que volvés si producción se rompe.

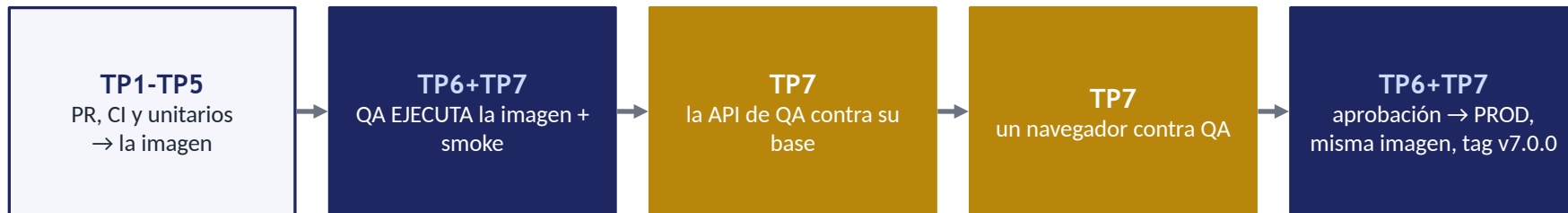
El rollback ya lo diste en la Clase 6, y acá no se practica. Lo que **SÍ** cambió es cómo se vuelve — y es la respuesta de la defensa:



Del tag a la imagen, en un paso: git rev-list te da el commit de v7.0.0, y ése es el sha- de la imagen que hay que desplegar. El comando completo, en la guía §3.5.

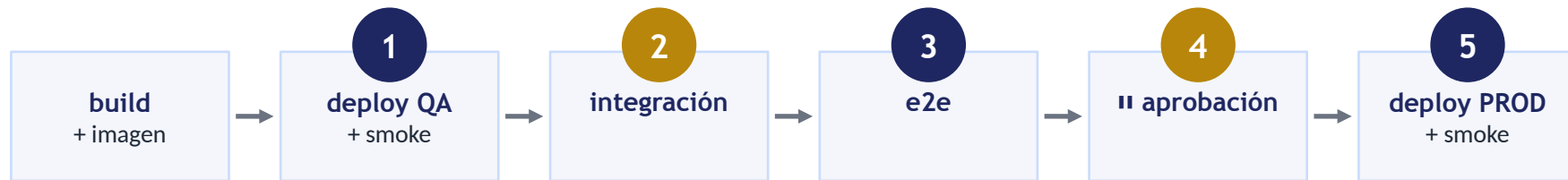
El sistema completo

viaja la imagen sha-abc123



Para pensar y saber contestar: ¿qué verifica cada control —unitario, smoke, integración, e2e— y qué NO puede ver? ¿Dónde se frena una promoción rota, y por qué ahí?

TP7: las cinco tareas, sobre la cadena



1 Cerrar la cadena

los cuatro servicios EJECUTAN la imagen que les nombra el pipeline

2 Integración contra QA

≥3 pruebas a la api de QA, sin navegador; cada una crea un dato que sólo es suyo, y lo borra al terminar

3 Suite e2e real

≥3 flujos contra QA, sin respuestas inventadas ni salteos

4 Las pruebas como gate

rojo rompiendo la app → PROD queda salteado

5 La release

v7.0.0 sobre el commit que está en PROD

TP7 – Entregables

Qué	Dónde, y qué lleva
1 • Formulario	Sólo la URL del repositorio público
2 • «Enlaces del TP7»	Arriba de todo en decisiones.md: (a) los dos paquetes (b) integración VERDE + e2e ROJA: el commit que rompió, la corrida y sus dos reportes (c) la corrida verde hasta PROD (d) las URLs de QA y PROD, VIVAS hasta la defensa
3 • decisiones.md	La sección del TP7, con los temas que lista la guía

Sin capturas: tu repositorio es público — Actions, los paquetes, Deployments y los Events de Render se navegan

Cierre — lo que se prueba es lo que se despliega

1

La cadena primero

Sin QA corriendo la imagen, lo demás prueba otra cosa

2

Próxima clase: infraestructura como código

Tu entorno, escrito en archivos: ver el plan antes de aplicar, recrearlo idéntico, detectar lo tocado a mano